

```

function [frequencies] = DecodeData(t,y)
% DecodeData
% -----
% Takes time-series data (t,y), sorts it, smooths it, interpolates it to a
% uniform time grid, performs an FFT, detects dominant frequency peaks, and
% returns up to the top 3 frequencies.
% -----
% --- Sort jumbled data (bubble sort) ---
[timeSorted, signalSorted] = mysortdata(t,y);
% --- Smooth signal (smaller window to preserve peaks) ---
[~, ySmoothed] = mysmoothdata(timeSorted, signalSorted, 25);
% --- Interpolate to uniform time steps of 0.001 s for better FFT resolution
% ---
[tInterpolated, yInterpolated] = myinterpolatedata(timeSorted, ySmoothed,
0.001);
% --- FFT calculation ---
N = length(yInterpolated); % number of samples
dt = tInterpolated(2) - tInterpolated(1); % sampling interval
Y = fft(yInterpolated); % compute FFT
% Two-sided spectrum
P2 = abs(Y/N);
% Single-sided spectrum (positive frequencies only)
P1 = P2(1:floor(N/2)+1);
P1(2:end-1) = 2*P1(2:end-1); % double non-DC components
% Frequency axis
frequency = (0:floor(N/2)) / (N*dt);
% Normalize power spectrum to max = 1
relativePower = P1 / max(P1);
% --- Plot results ---
subplot(1,2,1)
plot(tInterpolated, yInterpolated);
title('Original Signal');
xlabel('Time (Seconds)');
ylabel('Signal Amplitude (Arb. Units)');
grid on;
subplot(1,2,2);
plot(frequency, relativePower);
title('Power Spectrum');
xlabel('Frequency (Hz)');
ylabel('Normalized Power');
grid on;
% --- Detect peaks manually ---
peaks = [];
powers = [];
for i = 2:length(relativePower)-1
    % Local maxima + threshold + ignore DC region
    if relativePower(i) > relativePower(i-1) && ...
        relativePower(i) > relativePower(i+1) && ...
        relativePower(i) >= 0.1 && ...

```

```

        frequency(i) > 0.1

        peaks = [peaks, frequency(i)];    % store frequency
        powers = [powers, relativePower(i)]; % store corresponding power
    end
end
% --- Sort peaks by descending power (manual bubble sort) ---
for i = 1:length(powers)-1
    for j = i+1:length(powers)
        if powers(j) > powers(i)
            % swap power values
            tempP = powers(i); powers(i) = powers(j); powers(j) = tempP;
            % swap corresponding frequencies
            tempF = peaks(i); peaks(i) = peaks(j); peaks(j) = tempF;
        end
    end
end
% --- Keep maximum of 3 frequencies ---
frequencies = peaks(1:min(3,length(peaks)));
end
% =====
% Helper functions
% =====
function [time,signal] = mysortdata(time,signal)
% mysortdata
% Bubble-sort implementation to sort time and signal arrays together.
for i = 1:length(time)-1
    for j = 1:length(time)-1
        if time(j) > time(j+1)
            % Swap time
            t = time(j); time(j) = time(j+1); time(j+1) = t;
            % Swap signal to match
            s = signal(j); signal(j) = signal(j+1); signal(j+1) = s;
        end
    end
end
end
function [t_smoothed,y_smoothed] = mysmoothdata(t,y,NoPoints)
% mysmoothdata
% Simple moving-average smoothing with a specified odd window size.
t_smoothed = t;
y_smoothed = double(y);
% Ensure window size is odd
if mod>NoPoints,2) == 0
    NoPoints = NoPoints + 1;
end
n = (NoPoints - 1) / 2;
% Apply moving average
for i = 1+n:length(y)-n

```

```

    y_smoothed(i) = sum(y(i-n:i+n))/NoPoints;
end
end
function [t_interpolated,y_interpolated] = myinterpolatedata(t,y,t_step)
% myinterpolatedata
% Linear interpolation between each pair of points to create uniform spacing.
t_interpolated = t(1):t_step:t(end);
y_interpolated = zeros(size(t_interpolated));
for i = 1:length(t)-1
    % Find interpolated points between t(i) and t(i+1)
    idx = t_interpolated >= t(i) & t_interpolated <= t(i+1);
    y_interpolated(idx) = interpolate(t(i), t(i+1), y(i), y(i+1),
t_interpolated(idx));
end
end
function y_vals = interpolate(x1, x2, y1, y2, t_query)
% interpolate
% Linear interpolation formula between two known points.
slope = (y2 - y1) / (x2 - x1);
y_vals = y1 + slope * (t_query - x1);
end

```